PROJECT PROPOSAL

Incident Intelligence Platform

An evolving ASP.NET Core backend system — from a simple CRUD API to an AI-powered, event-driven, production-grade incident management platform.

Project Name	Incident Intelligence Platform
Project Type	Backend Engineering Roadmap / Portfolio Build
Core Stack	ASP.NET Core, EF Core, SQL Server, Redis, RabbitMQ, LLM/RAG
Delivery Model	10 Progressive Phases (Milestone-based)
Document Purpose	Proposal outlining phases, scope, and expected outputs

This proposal breaks the platform down into ten sequential phases. Each phase delivers a working, demonstrable improvement to the system while introducing a specific set of engineering skills — moving from basic CRUD operations through authentication, domain modeling, background processing, caching, event-driven architecture, AI-assisted root cause analysis, and finally full observability and production readiness.

Executive Summary

The Incident Intelligence Platform is a backend system designed to track, correlate, and eventually auto-diagnose operational incidents across a set of services. The system begins as a minimal CRUD API and evolves in ten well-defined phases into a distributed, event-driven, AI-assisted platform capable of automatically detecting incidents from logs, correlating them with deployments, and producing root cause analysis with supporting evidence.

Each phase is treated as an independent milestone rather than a fixed time-boxed sprint. A phase is considered complete when its stated outputs are functioning end-to-end, not when a calendar deadline is reached. This keeps the roadmap flexible while ensuring steady, visible progress.

Guiding Principle

Build only what the current phase needs. Advanced topics such as RabbitMQ, OpenTelemetry, and RAG are intentionally deferred until the foundation (CRUD, structure, auth, domain logic, and data) is solid. Phase 1 must be completed — a working Service and Incident CRUD API — before any later phase begins.

Roadmap at a Glance

#	Phase	Visible Result	Key Skills
1	Core Incident API	A working Incident API	ASP.NET Core, EF Core, DTOs
2	Professional API	Clean, structured API	Validation, Errors, Logging, Pagination
3	Authentication & Authorization	Multi-user system with roles	JWT, RBAC
4	Incident Timeline	Full incident history	Domain logic, State machines
5	Logs & Correlation	Logs linked to incidents	Querying, Correlation rules
6	Background Processing	Automatic incident detection	Hangfire, Async processing
7	Redis & Performance	Dashboard + caching	Redis, Cache-aside, TTL
8	Event-Driven Architecture	Decoupled async processing	RabbitMQ, Idempotency, Retries
9	AI Root Cause Analysis	Automated root cause reports	LLM APIs, RAG, Prompting
10	Observability & Production	Production-grade system	OpenTelemetry, Docker, Testing

ET Core Web API.

Entities

- **Service** — represents a monitored system component.
- **Incident** — Id, Title, Description, Severity, Status, CreatedAt, ResolvedAt, ServiceId.
- Relationship: one Service has many Incidents.

Endpoints

- POST /api/services · GET /api/services · GET /api/services/{id}
- POST /api/incidents · GET /api/incidents · GET /api/incidents/{id}
- PUT /api/incidents/{id} · DELETE /api/incidents/{id}

Skills Learned

- ASP.NET Core Web API, ControllerBase, Routing, IActionResult / ActionResult<T>
- DTOs, EF Core & DbContext, SQL Server, Dependency Injection
- LINQ basics, HTTP status codes

> *OUTPUT → A working Swagger-testable API supporting: create a Service → create an Incident → retrieve → update → resolve. This is the first functional version of the project.*

a real production project.

Architecture

- Layered flow: Controller → Service → Repository/EF Core → Database.

DTOs & Validation

- CreateIncidentRequest, UpdateIncidentRequest, IncidentResponse (no raw entities exposed).
- Validation rules: Title required, Description max length, Severity valid, ServiceId must exist.
- Implemented via FluentValidation or Data Annotations.

Cross-Cutting Concerns

- Centralized/global exception handling instead of per-controller try/catch (404 Not Found, 400 Validation Error, 500 Internal Server Error).
- Structured logging with Serilog.
- Pagination and filtering, e.g. GET /api/incidents?page=1&pageSize=20&severity=High&status=Open

OUTPUT → The API evolves from plain CRUD into a structured service with DTOs, validation, centralized error handling, logging, pagination, and filtering — a clear quality jump from Phase 1.

orization

e, multi-user system.

Users & Roles

- Roles: Admin, Developer, IncidentManager, Viewer.
- Viewer → read-only access. Developer → create/update incidents.
- IncidentManager → resolve incidents. Admin → full access.

Authentication Flow

- Register, Login, and Refresh Token flows implemented with JWT.

Endpoints

- POST /api/auth/register · POST /api/auth/login · POST /api/auth/refresh

> *OUTPUT → A complete Login → JWT → Role → Authorization → Incident API pipeline. The system is now a genuine multi-user application with role-based access control.*

ow into an object with real history.

Timeline Model

- New entity: IncidentEvent, capturing a timestamped history (created, severity changes, deployments detected, error spikes, assignment, resolution).

Endpoints

- GET /api/incidents/{id}/timeline · POST /api/incidents/{id}/events

Incident State Machine

- Enforced flow: Open → Investigating → Mitigated → Resolved.
- Invalid transitions are rejected (e.g. Resolved → Investigating, Open → Resolved) unless explicitly permitted by defined transition rules.

Skills Learned

- Domain logic, state machines, entity relationships, audit history, business rules.

> *OUTPUT → Opening an incident now shows full status, severity, and a chronological timeline — a clearly visible, user-facing feature.*

he real underlying problem.

Log Ingestion

- New entity: Log — Timestamp, Level, Message, ServiceId, TraceId.
- POST /api/logs · GET /api/logs
- Data model relationship: Incident → Service → Logs.

Simple Correlation Rules

- Deployment event + increased error rate → possible correlation flagged.
- 5xx error count exceeding a threshold → automatically create an Incident.

Note: no AI is introduced in this phase — this is the foundation the AI layer will later consume in Phase 9.

> *OUTPUT → Every Incident now carries structured context: its own Timeline, related Logs, and its associated Service.*

ng

t/response cycle.

Async Analysis Pipeline

- New logs feed a background job that analyzes service health and detects abnormal behavior.
- Implemented with Hangfire.
- Example recurring job (every 5 minutes): AnalyzeServices() checks error rate, latency, and existing incidents per service, and creates new incidents automatically when needed.

Reliability Features

- Retry logic for failed jobs, scheduled jobs, and job execution history.

> *OUTPUT → Incidents can now be detected automatically — the user no longer has to manually create every incident. This is a major functional leap for the platform.*

oad.

Caching Strategy

- Cache-aside pattern applied to hot endpoints such as GET /api/services and GET /api/incidents/statistics.
- Flow: Request → Redis → Cache Hit (return) / Cache Miss → SQL Server → populate cache.

Dashboard Statistics

- Example metrics: Open Incidents, Critical Incidents, Resolved Today, Most Affected Service.

Skills Learned

- Redis, cache-aside pattern, TTL, cache invalidation, query optimization, database indexes.

OUTPUT → The system becomes noticeably faster, gains dashboard-ready statistics, and is more scalable.

ture (RabbitMQ)

— first step into distributed systems.

From Synchronous to Event-Driven

- Before: POST /logs → analyze → detect → create incident, all inline.
- After: POST /logs → save → publish LogReceived event → RabbitMQ → Analysis Worker consumes → analyzes → creates Incident.

Core Events

- LogReceived, IncidentCreated, IncidentResolved, DeploymentDetected.

Distributed Systems Concepts

- Producers and consumers, retry handling, Dead Letter Queues, duplicate message handling, idempotency, eventual consistency.
- Key scenario to solve: if RabbitMQ delivers the same event twice, the consumer must check "have I processed this before?" and ignore duplicates instead of reprocessing them.

> *OUTPUT → Log processing is fully decoupled from the API layer, with resilient, idempotent event consumption — a real encounter with distributed systems challenges.*

is

has real context.

By this phase the platform already has Incidents, Logs, Timelines, Deployments, Services, and Events — meaning there is genuine, structured context for an AI model to reason over.

Analysis Pipeline

- Input: Incident + relevant Logs + recent Deployments + Timeline.
- Output: an AI-generated Root Cause, supporting Evidence, a Confidence score, and Recommended Actions.

Example Output

Root Cause: Database connection pool exhaustion.

Evidence:
• 82% of failed requests contain a DB timeout.
• Error rate increased 4 minutes after deployment v42.
• DB connection utilization reached 98%.

Confidence: 87%

Recommended Actions:
1. Investigate deployment v42.
2. Increase connection pool size.
3. Check long-running queries.

Approach

- Start with a direct LLM API integration.
- Progress to Retrieval-Augmented Generation (RAG) so the AI retrieves relevant logs/events before analysis.

OUTPUT → The platform automatically produces root cause reports with supporting evidence and recommended actions for real incidents.

ction Readiness

ion-grade.

Observability Stack

- OpenTelemetry collecting Logs, Metrics, and Traces.
- Correlation ID / Trace ID / Request ID propagated across the request → API → RabbitMQ → Worker → SQL Server pipeline, enabling full failure tracing.

Production Hardening

- Health checks, rate limiting, retry policies, structured logging.
- Docker containerization.
- Integration tests and API tests.

OUTPUT → A complete backend system: Client → API → Database, with Redis caching, RabbitMQ-driven async workers, full observability, and an AI root-cause layer on top — ready for production.

Delivery Philosophy

No phase is treated as a fixed-duration sprint. Each is a milestone, reached when its output works end-to-end:

Phase	Milestone Statement
1	“I can build a CRUD API on my own.”
2	“I can build a well-structured API.”
3	“I can implement authentication.”
4	“I can model a business workflow.”
5	“I can work with real, correlated data.”
6	“I can run processing outside the request cycle.”
7	“I can improve system performance.”
8	“I understand message queues.”
9	“I can integrate AI into a real system.”
10	“I can handle production concerns.”

Each completed phase adds a real, working capability to the system while building a specific, transferable engineering skill — ensuring the project is both a functioning product and a structured learning path.

Immediate next step: build Phase 1 only. Advanced topics such as RabbitMQ, OpenTelemetry, and RAG are deliberately out of scope until the Service and Incident CRUD API is complete.